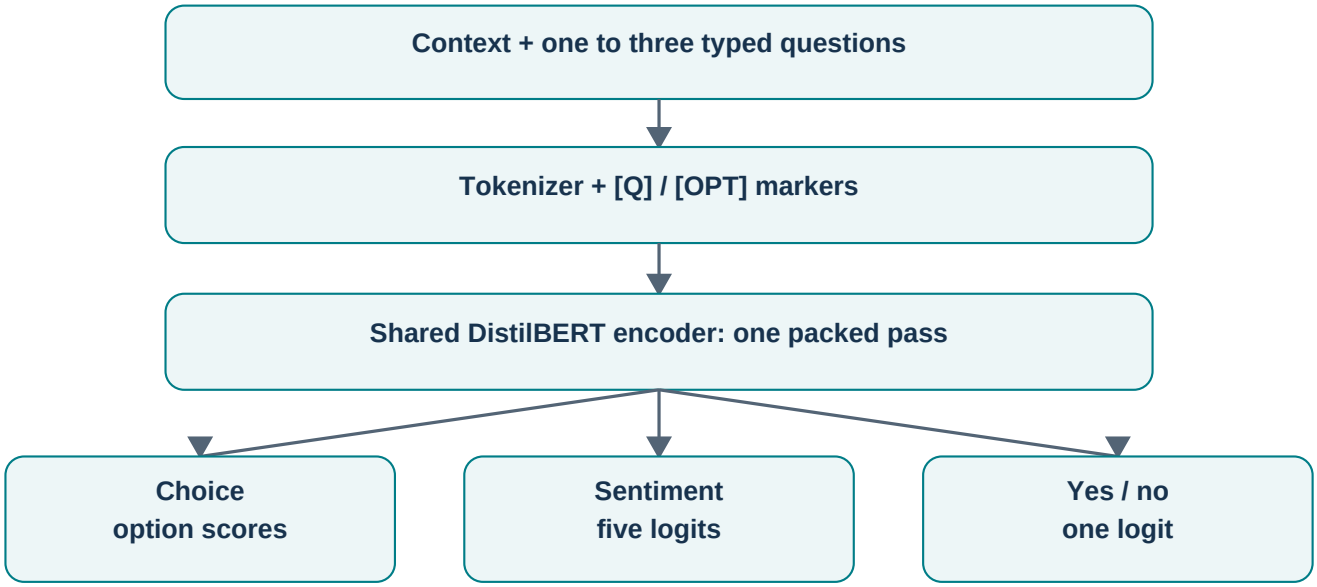
Jevlet

How one shared encoder turns text and questions into typed decisions

A practical guide to the implementation, training process, probability outputs, and real predictions from the selected checkpoint.



Three useful answers, one interface

Choice: choose a banking intent from the supplied options.
Score: assign an English review a sentiment level from 1 to 5.
Yes/no: decide whether a banking message concerns a queried intent.

The context is written once in a packed sequence. Each question has its own marker, so its answer can be read from the corresponding encoder representation. The heads return probabilities and a typed answer rather than generated prose.

Authors: Hitanshu Gala, Jay Dalvi, Satishkumar Varma, and Richa Sharma
Dwarkadas J. Sanghvi College of Engineering, Mumbai, India

From a request to a packed sequence

1. Validate the request

The implementation accepts one to three questions. Every question needs a unique ID, a type, and nonempty text. Choice questions need two to ten distinct options. Literal special-token strings in user input are removed so they cannot create extra markers.

2. Add markers and tokenize

```
[CLS] [Q] Which intent? [OPT] lost card
[OPT] cash withdrawal [Q] Is this about lost card?
[SEP] I lost my debit card. [SEP]
```

This schematic shows two questions, not the full model input for every request. The tokenizer turns words into subword token IDs and records the positions of each [Q] and [OPT] marker.

3. Preserve the question block

The input budget is 256 tokens. Questions and options are retained first; only the trailing context is truncated if necessary. The response includes `context_tokens_dropped` so the application can see when evidence was shortened. A question block that leaves no room for context is rejected.

4. Encode the sequence once

DistilBERT-base has six transformer layers, hidden width 768, and 12 attention heads. Self-attention allows question and option markers to interact with the shared text. The implementation gathers marker vectors from the final hidden states. It does not independently encode every option.

Vector read	Head receiving it
Each [OPT] representation	Shared linear option scorer
[Q] representation for sentiment	Five-class sentiment head
[Q] representation for a predicate	Binary head

Packed requests can save repeated encoder work. They also share the token budget: adding long questions can leave less space for context. Sharing a context is not a guarantee of lower latency than every conventional classifier.

How each head predicts

Choice: compare only the supplied options

One linear layer maps each option-marker vector to a scalar logit. A mask excludes unused option slots. Softmax converts the candidate logits into probabilities that sum to one; the highest-probability option becomes the answer.

```
p(option j) = exp(logit j / T) / sum exp(logit k / T)
```

The result is relative to this shortlist. The training/evaluation shortlist contains the correct label; accuracy does not establish retrieval of the correct intent from all 77 possible labels.

Sentiment: five levels and an expected score

The selected categorical head maps a question-marker vector to five logits. Softmax produces a probability for each level. The winning index plus one gives the discrete answer. The expected score is the probability-weighted average of levels 1 to 5.

Level	Meaning
1	Very negative
2	Negative
3	Neutral
4	Positive
5	Very positive

An earlier variant uses a scalar and four ordered thresholds to form cumulative probabilities. The five-class version directly optimizes class discrimination; it does not impose an ordering constraint on its logits.

Yes/no: a probability of the predicate being true

A linear layer produces one logit. Sigmoid gives P(yes); P(no) is one minus that value. The higher probability determines the boolean answer. A confidence of 0.9 means the model assigns that answer 90% probability; it is not proof that the answer is correct.

Temperature T is positive and fitted separately for each head on calibration data. It changes confidence. Positive temperature scaling preserves categorical and binary argmax decisions; it can change the ordinal head's winning level.

Training, validation, and calibration

Partition	Banking texts	Review texts	Purpose
Training	7,993	8,531	Learn encoder and head weights
Validation	999	550	Select checkpoint/settings
Calibration	1,000	550	Fit confidence temperatures
Test	3,079	2,210	Report final quality

Learning from labels

Training derives intent choices, binary intent predicates, and review sentiment questions from public dataset labels. Cross-entropy supervises choice and categorical sentiment; binary cross-entropy supervises yes/no. A Brier penalty of weight 0.5 encourages useful probability distributions. Losses are averaged within each present head and then across those heads.

Shared learning

Backpropagation updates the task heads and shared encoder. A change that helps sentiment can alter banking predictions because both tasks use the same backbone. GPU training uses mixed precision, gradient accumulation, AdamW, and gradient clipping; evaluation uses float32.

What the expanded validation changes

Five fixed question views per validation text produce 7,745 requests and 17,735 decisions. Banking views vary wording, candidates, and queried predicates. Review views vary the wording of the same five-level sentiment task. Training also varies question wording. There are still only 1,549 unique validation contexts: repeated views are correlated, not new independent labels.

Three refinement settings

The search continues from the previous five-class checkpoint using three seed/learning-rate settings, for at most three epochs each. Expanded-validation sentiment accuracy chooses the winner, subject to a one-percentage-point banking regression limit. The original checkpoint remains an eligible fallback. Candidate test scores are not used for ranking.

Calibration is separate

After training, one temperature per head minimizes calibration-set negative log-likelihood. The test split is evaluated only after selection. Earlier test results have already been inspected, so this study remains exploratory and needs fresh-data confirmation.

Real predictions from the checkpoint

These hand-written examples are qualitative demonstrations, not extra benchmark samples. Every probability below was obtained by running the selected Jevlet checkpoint on CUDA.

Banking: several questions share one message

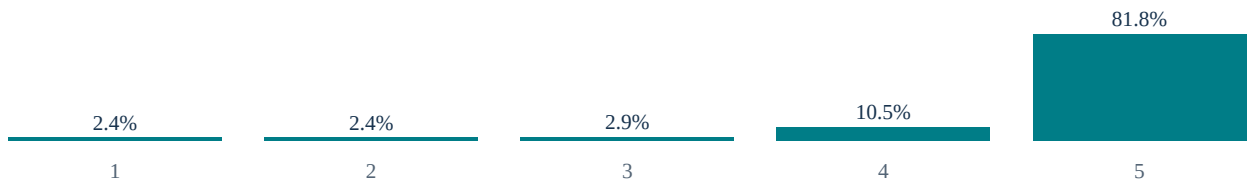
I lost my debit card and need to order a replacement.

Question	Answer	Confidence
What is the customer's intent?	lost or stolen card	99.6%
Is this about lost or stolen card?	True	98.7%
Is this about cash withdrawal?	False	99.0%

Review: five-level sentiment

The performances are wonderful, and the story is moving and beautifully told.

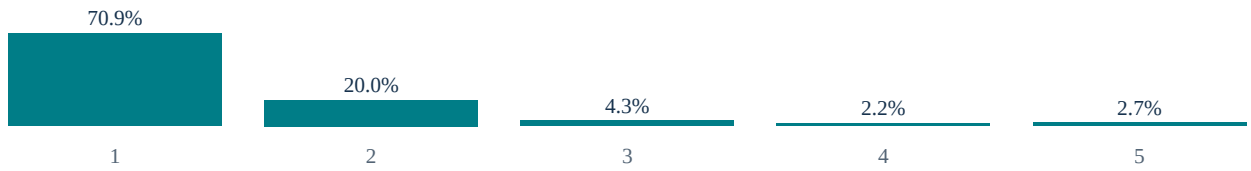
Predicted level: **5** | Expected score: **4.67** | Confidence: **81.8%**



Review: five-level sentiment

This movie is dull, badly acted, and a complete waste of time.

Predicted level: **1** | Expected score: **1.46** | Confidence: **70.9%**



A review can receive a confident wrong answer. Discrete accuracy, macro-F1, ordinal error, and calibration measure different aspects of prediction quality. Inspecting a few examples cannot replace held-out evaluation.

What the results establish

Selected checkpoint: calibrated test	Measurement
Banking shortlist accuracy	95.97%
Sentiment accuracy	51.72%
Sentiment macro-F1	0.4929
Sentiment mean absolute error	0.5674
Binary intent accuracy	97.08%

What to look at in a response

answer: the chosen option, level, or boolean.
probabilities: uncertainty across the answer space.
confidence: the probability of the winning answer.
expected_score: a continuous sentiment estimate for score questions.
context_tokens_dropped: evidence removed by truncation.

Where Jevlet is useful

A single compact checkpoint can answer several bounded questions over the same text, with a consistent probability interface. It supports application logic that needs typed decisions. The evidence covers known English banking intents and five-level review sentiment, rather than arbitrary natural-language reasoning.

Where caution is needed

Unseen options, question paraphrases outside the tested templates, different languages, missing gold labels, and new domains are not established capabilities. A score question about banking sentiment was not supervised by the review labels. Refinements share an initial checkpoint, and repeated validation views do not establish cross-validation or independent training-seed robustness.

Implementation map

`data.py`: sampling, markers, token packing.
`model.py`: shared encoder, typed heads, prediction.
`train.py`: learning and checkpoint selection.
`metrics.py`: calibration and quality measures.
`refine.py`: expanded-validation refinement search.

Sources: the local Jevlet implementation and measured run artifacts; DistilBERT (arxiv.org/abs/1910.01108); BANKING77 (aclanthology.org/2020.nlp4convai-1.5/); SST (aclanthology.org/D13-1170/); temperature scaling (proceedings.mlr.press/v70/guo17a.html).